

# HireMind AI

## Multi-Agent Resume Screening & Job Matching Platform FINAL PROJECT REPORT — Hackathon Problem Statement PS03

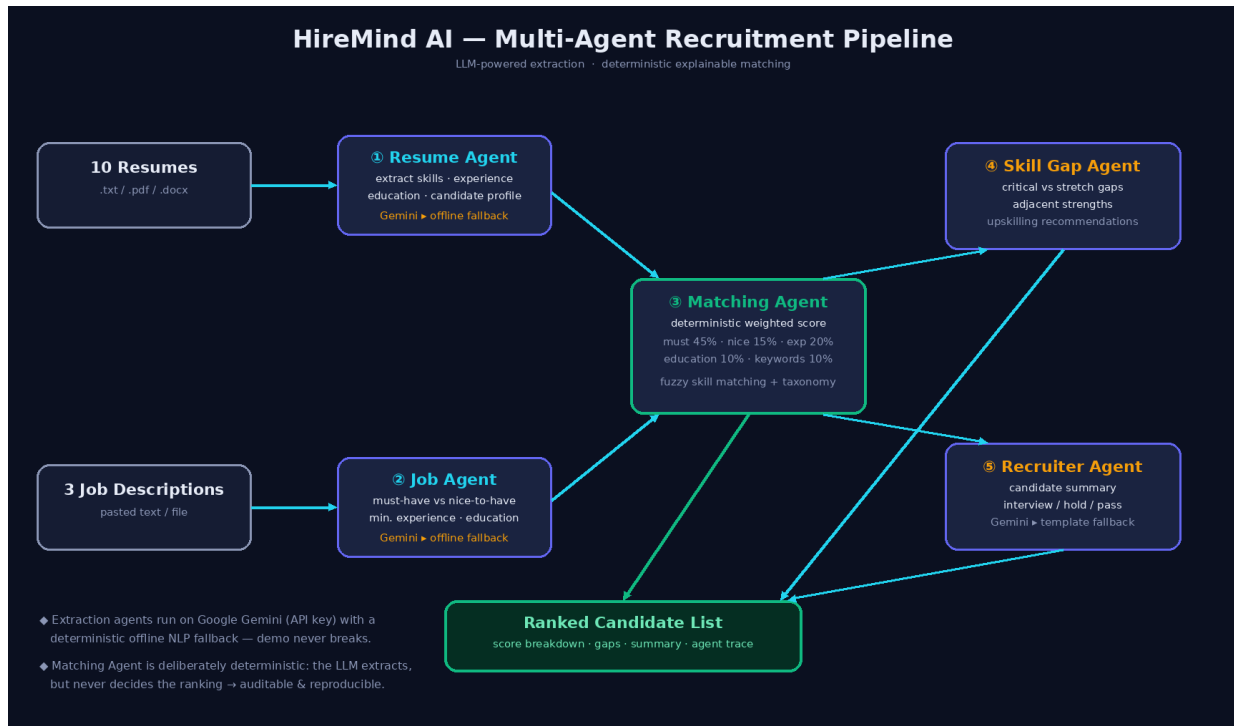


Figure 0 — Five-agent pipeline: LLM-powered extraction around a deterministic, explainable matching core.

| Field               | Details                                                     |
|---------------------|-------------------------------------------------------------|
| Problem Statement   | PS03 — Multi-Agent Resume Screening & Job Matching Platform |
| Technology Category | NLP + LLM + Agentic AI                                      |
| Team                | [YOUR TEAM NAME] — [MEMBERS]                                |
| Hackathon           | [HACKATHON NAME]                                            |
| Date / Version      | 20 Sept 2026 · v2.0 (final)                                 |

# 1. Executive Summary

HireMind AI is a multi-agent recruitment platform that screens resumes against job descriptions and produces ranked, fully explainable candidate–role matches. Five cooperating agents — Resume, Job, Matching, Skill Gap and Recruiter — mirror a human recruiting team at machine speed.

Architectural signature: **“LLM extracts, never decides.”** Google Gemini (optional, user-supplied key) performs information extraction and summarisation; the ranking decision is produced by a deterministic weighted engine, giving reproducible, auditable outcomes. A rule-based offline NLP layer makes the platform fully functional without any API key.

The intelligence layer is framework-agnostic: the identical agent codebase is served by **two interchangeable backends** — Python FastAPI and Django — behind one REST contract, with a React 18 frontend on top. The PS03 required demonstration (10 resumes × 3 job descriptions → ranked lists) passes end-to-end on both, verified by a 72-check automated QA suite (72/72 passing).

# 2. Problem Analysis

Manual resume screening is slow (≈7 seconds per resume at first pass), inconsistent between screeners, vocabulary-sensitive (“React.js” vs “React”), and opaque to candidates. We decomposed the problem into five sub-problems, each assigned to one agent:

| Sub-problem                                               | Agent responsible                |
|-----------------------------------------------------------|----------------------------------|
| P1 Heterogeneous resume formats (txt/pdf/docx, free text) | Resume Agent                     |
| P2 Ambiguous JDs (must-have vs nice-to-have separation)   | Job Agent                        |
| P3 Multi-dimensional compatibility measurement            | Matching Agent                   |
| P4 Actionable feedback on missing skills                  | Skill Gap Agent                  |
| P5 Explainability, reproducibility & trust                | Deterministic core + Agent Trace |

# 3. Research

## 3.1 Approaches surveyed

- Keyword/ATS matching — fast, brittle to synonyms.
- Embedding similarity — synonym-tolerant but opaque single score.
- LLM-only screening — expressive but non-reproducible and hard to audit.
- Hybrid agentic systems (2025–26 SOTA) — specialised agents around a deterministic decision core; adopted.

## 3.2 Design conclusions

- Extraction = language task → Gemini with deterministic fallback; all outputs normalised through a shared ~130-skill taxonomy (aliases: js→javascript, k8s→kubernetes, sklearn→scikit-learn).
- Ranking = decision task → published fixed weights; fuzzy matching (containment + difflib ≥ 0.85) removes vocabulary false-negatives.

## 3.3 Datasets

Kaggle/Hugging Face resume & JD corpora surveyed for scale-up; demonstration uses a fully synthetic 10+3 dataset (no participant data, per PS03). Any real dataset adopted later will have license/version/access conditions verified first, as the source note requires.

## 4. Proposed Solution

| # | Agent           | Output                                                       | Engine                                |
|---|-----------------|--------------------------------------------------------------|---------------------------------------|
| 1 | Resume Agent    | Profile: canonical skills, experience, education, highlights | Gemini JSON-mode / offline heuristics |
| 2 | Job Agent       | Must-have / nice-to-have skills, min experience, education   | Gemini JSON-mode / offline heuristics |
| 3 | Matching Agent  | 0–100 score, verdict, per-dimension breakdown                | Deterministic (always)                |
| 4 | Skill Gap Agent | Critical & stretch gaps, adjacent strengths, learning plan   | Deterministic rules                   |
| 5 | Recruiter Agent | Summary + interview/hold/pass recommendation (parallelised)  | Gemini / template                     |

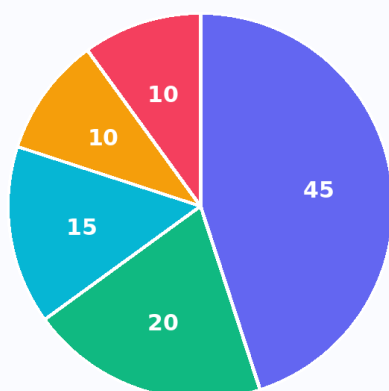
- Dual-mode resilience: identical schema from Gemini or offline path — the demo never breaks on stage.
- Single REST contract implemented twice (FastAPI, Django) over one shared agent codebase.
- Agent Trace: every match carries each agent's engine + full JSON output for audit.

## 5. Technical Flow & Scoring Model

- Intake → Extraction (Agents 1–2) → Weighted scoring (Agent 3) → Gap analysis (Agent 4) → Summary (Agent 5) → Ranked list + trace.

| Dimension           | Weight | Computation                                                |
|---------------------|--------|------------------------------------------------------------|
| Must-have skills    | 45%    | Fraction matched (fuzzy + taxonomy)                        |
| Nice-to-have skills | 15%    | Fraction matched                                           |
| Experience          | 20%    | 1.0 if ≥ required; 0.6×(cand/req) below; 0.75 unspecified  |
| Education           | 10%    | Degree-rank comparison (PhD > Master > Bachelor > Diploma) |
| Keyword affinity    | 10%    | JD keywords found in resume text                           |

### How Score Bot grades — like a test with clear rules



#### The score pie:

- Must-have skills — 45% of the score
- Experience — 20% of the score
- Nice-to-have skills — 15% of the score
- School / degree — 10% of the score
- Bonus keywords — 10% of the score

Same resume + same job =  
same score. **ALWAYS. (fair!)**

Figure 1 — The score pie: fixed, public slices; same input  $\Rightarrow$  same score, always.

Verdicts:  $\geq 80$  Strong · 60–79 Good · 40–59 Partial ·  $< 40$  Weak. Weights renormalise when a dimension is absent.

## 6. Technology Stack

| Layer       | Technology                                        | Rationale                                              |
|-------------|---------------------------------------------------|--------------------------------------------------------|
| Frontend    | React 18 + Vite                                   | Agent cards, ranked lists, explainable panels; HMR dev |
| Backend A   | Python 3 + FastAPI (Uvicorn)                      | Async REST, OpenAPI docs, serves React build           |
| Backend B   | Django 6 (runserver/WSGI)                         | Same API contract; proves framework-agnostic agents    |
| LLM         | Google Gemini 2.5 Flash (REST)                    | JSON-mode extraction/summaries; user-supplied key      |
| Offline NLP | Regex + 130-skill taxonomy + difflib              | Deterministic fallback & alias normalisation           |
| Parsing     | pypdf · python-docx                               | Real-world resume formats                              |
| QA          | test_pipeline.py · test_everything.py (72 checks) | Automated verification of both backends                |

### 6.1 Repository layout

```
backend/ FastAPI app + agents/{llm,resume,job,matching,skill_gap,recruiter} + skills_taxonomy +  
offline_extract + store + static/ (React build)  
backend_django/ Django project (manage.py, hiremind/{settings,urls,views}) reusing backend/  
agents via sys.path  
frontend/ React + Vite source (src/components/*, api.js)  
data/resumes/ (10) · data/jobs/ (3) synthetic demo dataset  
docs/ reports (PDF/DOCX), pitch deck (PPTX), diagrams, screenshots  
test_pipeline.py · test_everything.py
```

## 7. REST API (identical on both backends)

| Endpoint                                                 | Purpose                                               |
|----------------------------------------------------------|-------------------------------------------------------|
| POST /api/resumes · /api/resumes/upload                  | Register resumes (paste or multipart files) → Agent 1 |
| POST /api/jobs                                           | Register JD → Agent 2                                 |
| POST /api/demo/load                                      | Load 10-resume / 3-JD demo set                        |
| POST /api/match {job_id}                                 | Agents 3–5 over all candidates → ranked list + traces |
| POST /api/reextract · POST /api/settings                 | Re-extraction with Gemini · key/model config          |
| GET /api/agents · /api/health · /api/resumes · /api/jobs | Metadata, health (reports backend flavour), listings  |
| DELETE /api/resumes/{id} · /api/jobs/{id} · /api/reset   | Housekeeping                                          |

## 8. Application Screenshots (captured live)

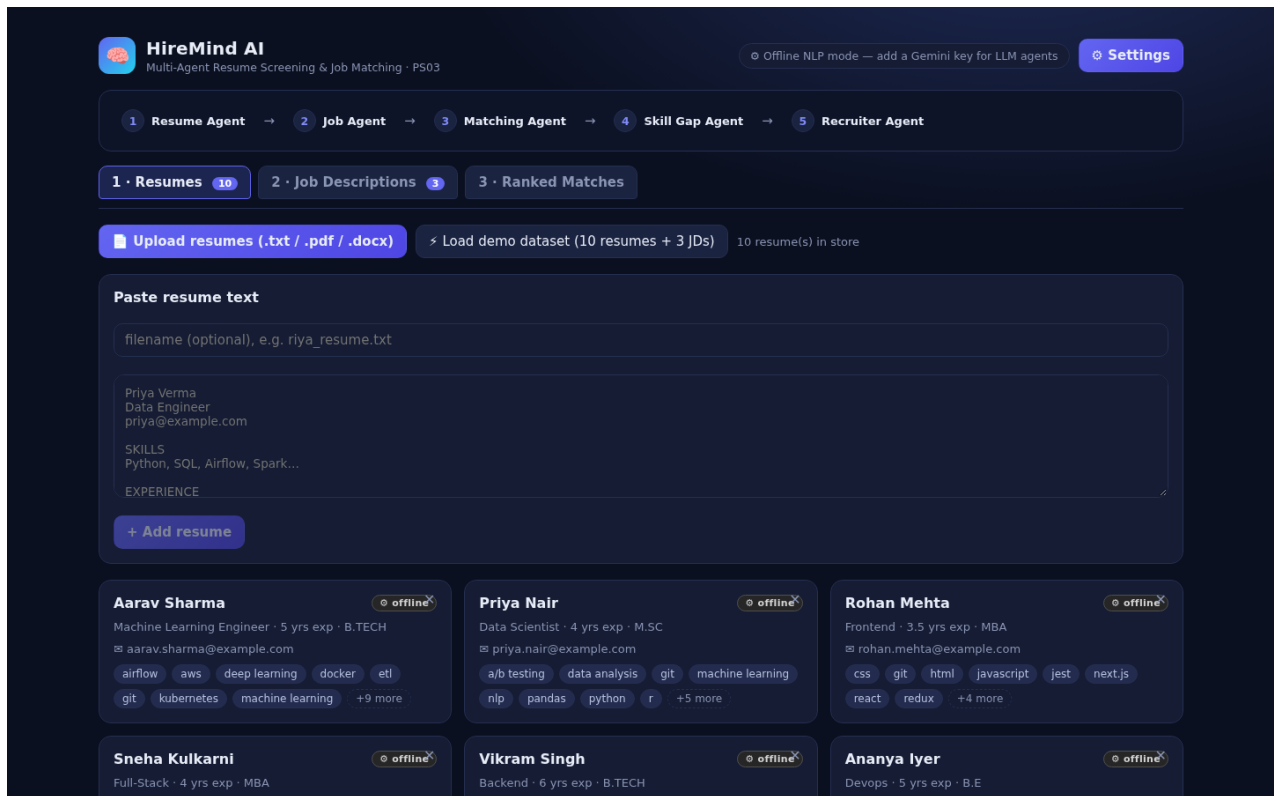


Figure 2 — Resume Agent output: 10 extracted candidate cards (skills, experience, education, engine badge).

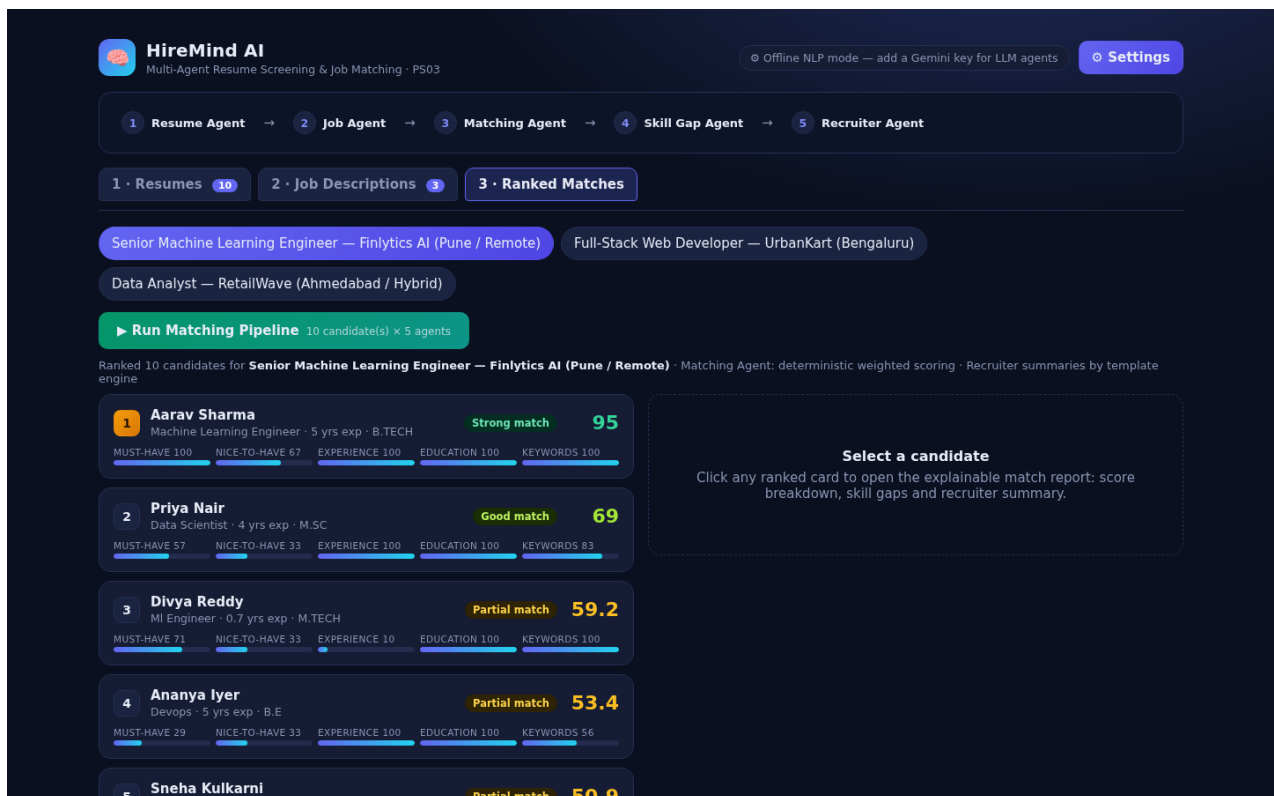


Figure 3 — Ranked list for Senior ML Engineer with verdict badges and per-dimension score bars.

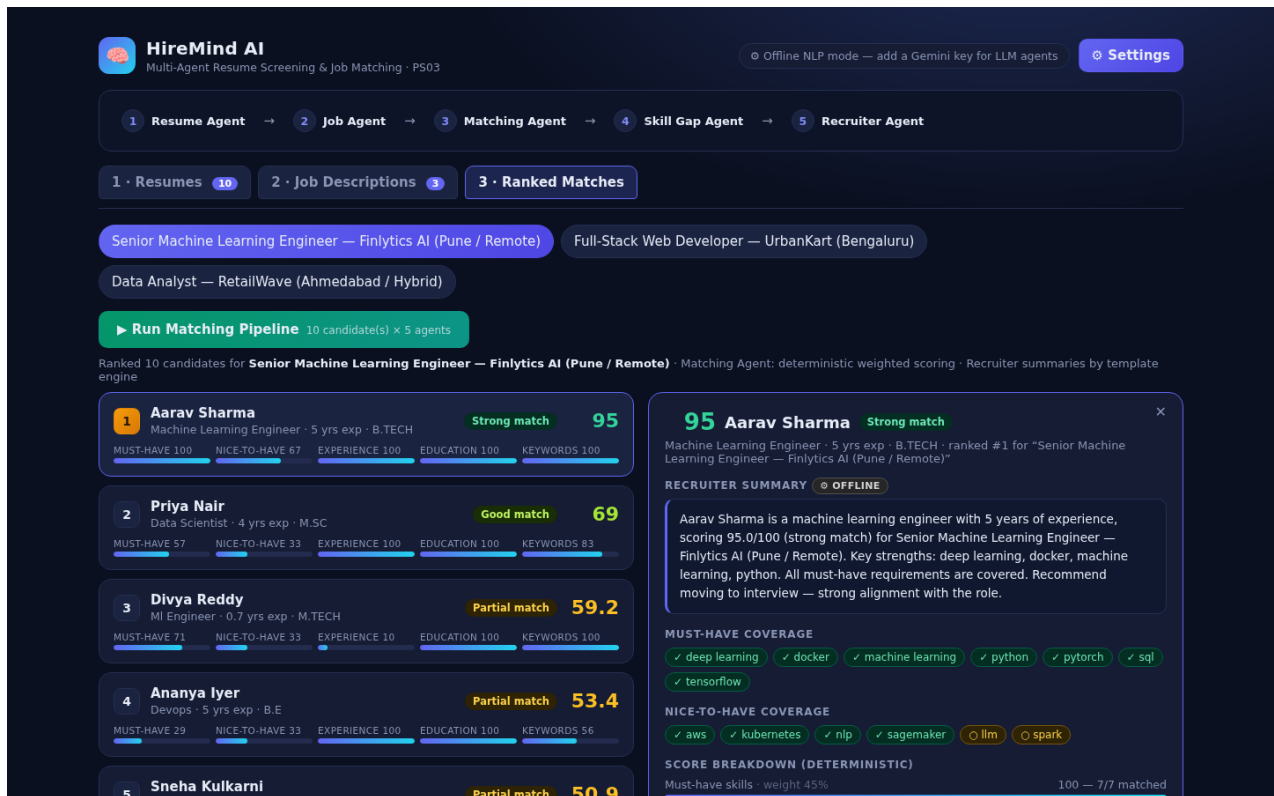


Figure 4 — Explainable match report: recruiter summary, ✓/✗ coverage, weighted breakdown, gap plan, agent trace.

## 9. Verification & QA (72/72 passing)

Two suites run on **both** backends: test\_pipeline.py (end-to-end demo) and test\_everything.py (25 behaviour checks per backend + 22 deliverable-file checks). Latest run: **72 passed, 0 failed**.

| Job                  | #1                                  | #2                        | #3                          |
|----------------------|-------------------------------------|---------------------------|-----------------------------|
| Senior ML Engineer   | Aarav Sharma — 95.0 Strong (7/7)    | Priya Nair — 69.0 Good    | Divya Reddy — 59.2 Partial  |
| Full-Stack Developer | Sneha Kulkarni — 100.0 Strong (5/5) | Rohan Mehta — 81.0 Strong | Vikram Singh — 66.2 Good    |
| Data Analyst         | Karan Patel — 80.7 Strong           | Priya Nair — 78.0 Good    | Aarav Sharma — 46.7 Partial |

- Identical rankings on FastAPI and Django — the agent layer is provably framework-independent.
- Reproducibility: repeated runs produce byte-identical scores and ordering (deterministic core).
- Error paths verified: unknown job/resume → 404; empty inputs → 400; reset/reextract flows pass.
- UI verified by headless-Chromium screenshots; PDF report pages rendered and visually inspected.

## 10. Explainability, Fairness & Innovation

- Published weights + plain-language notes per dimension; Agent Trace exposes every agent's raw I/O.
- Scoring uses only skills/experience/education/keywords — never names, photos, ages, genders.
- Alias + fuzzy matching removes terminology bias; deterministic core removes run-to-run variance.
- Innovations: “LLM extracts, never decides”; dual-engine resilience; adjacency-aware gap analysis with learning plans; shared taxonomy as inter-agent contract; dual-backend portability; one-click PS03 demo.

## 11. Limitations & Roadmap

---

- Offline heuristics extract fewer skills from very unstructured resumes (mitigated by dual mode).
- In-memory persistence → roadmap: SQLite/Postgres, match history, accounts.
- Roadmap: embedding similarity as 6th signal; layout-aware PDF parsing; multi-job comparison board; CSV/PDF export; scheduled fairness audits.

## 12. External / AI Resources (Disclosure)

---

- Google Gemini 2.5 Flash — optional runtime extraction/summaries via official REST API (user key, never hard-coded).
- AI-assisted development — Arena.ai Agent Mode assisted scaffolding/drafting; all design decisions and verification team-reviewed.
- Open-source: React, Vite, FastAPI, Django, python-docx, pypdf, Pillow, reportlab, python-pptx, Playwright, difflib.
- Demo data 100% synthetic; no participant data collected (per PS03).

## 13. References

---

- [1] Gemini API — [ai.google.dev](https://ai.google.dev) · [2] FastAPI — [fastapi.tiangolo.com](https://fastapi.tiangolo.com) · [3] Django — [djangoproject.com](https://djangoproject.com)
- [4] React — [react.dev](https://react.dev) · [5] Vite — [vite.dev](https://vite.dev) · [6] pypdf / python-docx docs
- [7] Python difflib — [docs.python.org](https://docs.python.org) · [8] Kaggle & Hugging Face resume/JD datasets (surveyed; verify licenses)

## Appendix A — Run & demo

```
# FastAPI: cd backend && pip install -r requirements.txt && uvicorn main:app --host 0.0.0.0 --port 8000
# Django: pip install django && cd backend_django && python manage.py runserver 0.0.0.0 --port 8001
# Frontend dev: cd frontend && npm install && npm run dev # /api proxied to :8000
# QA: python3 test_everything.py # 72 checks across both backends
Demo: open :8000 or :8001 → Load demo dataset → Tab 3 → Run Matching Pipeline → open any candidate → Agent Trace.
```